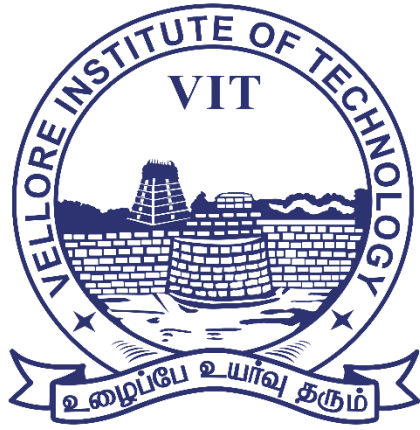




VIT[®]
B H O P A L
www.vitbhopal.ac.in

JAVA Project

Food Order Management System

Name	Pratham Yadav
REG no.	25BAI11032
Course code	CSE2006
Course Name	Programming in java

1. INTRODUCTION

The **Food Order Management System** is a console-based Java application that simulates the

complete backend operations of a digital food ordering platform. It is engineered entirely using core

Java Standard Library features — no external databases, frameworks, or dependencies are required.

The system demonstrates enterprise-level software architecture through a 4-layer decoupled design,

the Singleton design pattern, interface-driven business logic, custom exception handling, Java Streams,

and strict input validation via Regular Expressions — all concepts covered under CSE2006.

2. PROBLEM STATEMENT

Managing food orders manually in environments like campus cafeterias leads to recurring operational

problems: missed orders, billing errors, and no transaction history. While enterprise web applications

solve this, they demand SQL servers, JDBC drivers, and complex network configurations unsuitable for

lightweight academic deployments.

This project addresses the need for a portable, zero-dependency Java application capable of

managing secure user sessions, dynamic menus, shopping cart processing, invoice generation, and

order history — entirely within the Java Virtual Machine (JVM).

3. FUNCTIONAL REQUIREMENTS

3.1 User Management Module

- Register new users with unique email validation using Java Regex
- Simulate password security via Base64 encoding before storage
- Login with email + password credential matching
- Throw `UserNotFoundException` for invalid login attempts
- Restrict all ordering features until successful authentication

3.2 Restaurant & Menu Catalogue Module

- Display a pre-seeded list of all restaurants with cuisine types
- Filter menus dynamically — only items where `isAvailable = true` are shown
- Look up a specific menu item by its unique ID via `RestaurantService`
- Display menus in tabular printf-formatted layout

3.3 Order Processing Module

- Add menu items to a session-based cart by item ID
- Throw `ItemNotAvailableException` if item is null or unavailable
- Group identical items by quantity using overridden `equals()` + `hashCode()`
- Calculate cart total using Java Streams (`mapToDouble`)
- Confirm order with Y/N prompt — cart preserved on cancellation
- Generate time-stamped invoice using `LocalDateTime`
- Archive confirmed orders to `InMemoryDB` order ledger
- Retrieve per-user order history from the in-memory database

4. NON-FUNCTIONAL REQUIREMENTS

Requirement Implementation

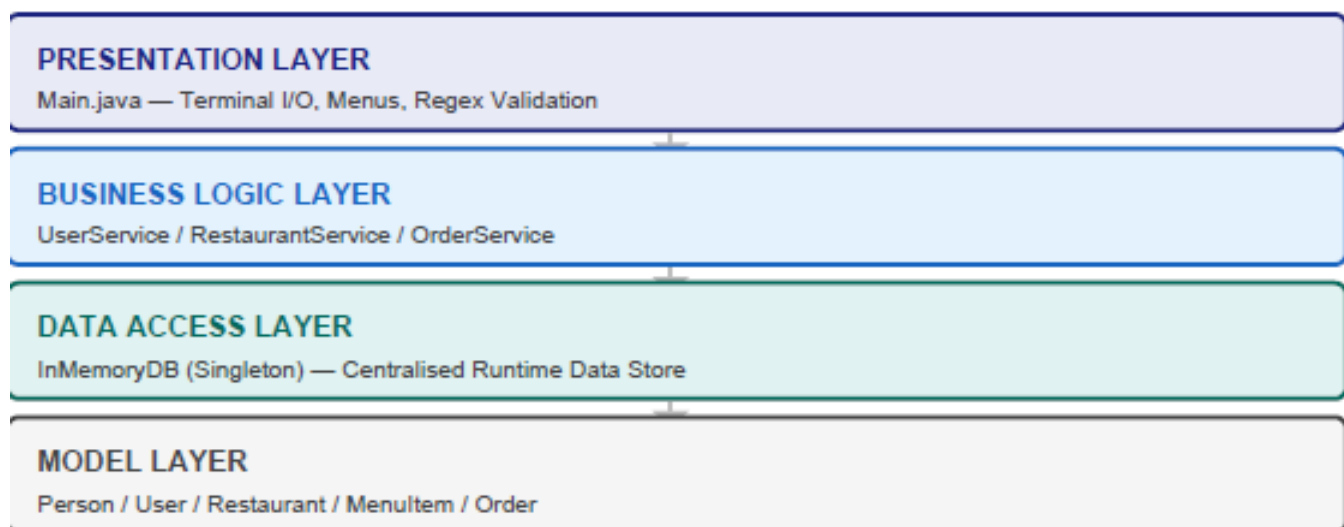
Portability	Runs on any JVM (JDK 17+) — no SQL drivers, external libraries, or network config required
Performance	Runs on any JVM (JDK 17+) — no SQL drivers, external libraries, or network config required
Reliability	Custom exceptions + recursive Scanner buffer clearing prevent any runtime crash from bad input
Security	Regex validates all inputs; passwords are Base64 encoded before storage; DB fields are private
Usability	ANSI colour codes, printf table formatting, and screen-pacing pauses for professional CLI UX
Maintainability	4-layer package architecture (model/service/exception/database) — single responsibility per class
Encapsulation	InMemoryDB lists are private with unmodifiable getters; writes only via addUser() / addOrder()

5. SYSTEM ARCHITECTURE

The project follows a **4-layer decoupled architecture** separating concerns across four distinct

packages. Each layer communicates only with the layer directly below it, ensuring changes in one layer

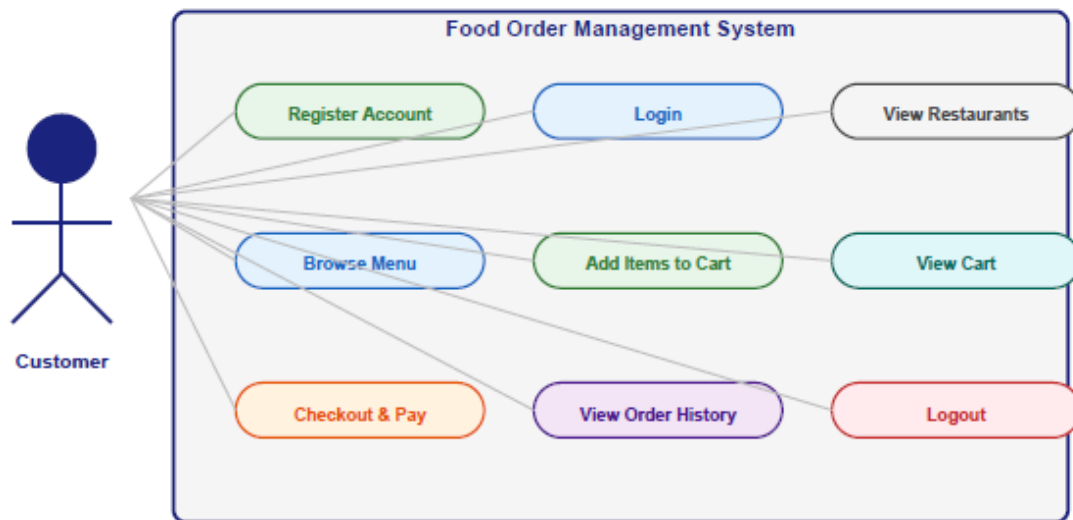
do not cascade to others.



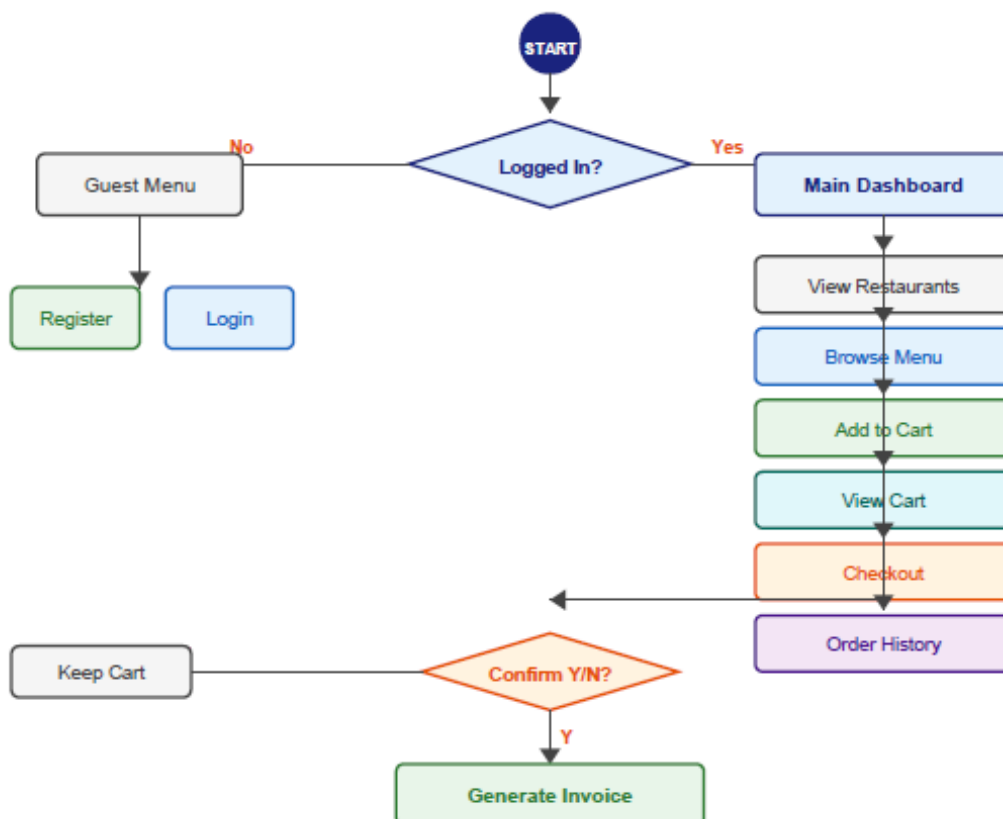
Main.java (root)	Terminal I/O, menus, Regex validation, routing, ANSI formatting
Database/InMemoryDB.java	Singleton data store — private lists, unmodifiable getters, addUser()/addOrder()
model/Person.java	Abstract base class — id, name, abstract getDetails()
model/User.java	Extends Person — email, passwordEncoded, overrides getDetails()
model/Resturant.java	Restaurant entity — id, name, cuisine, toString()
model/MenuItem.java	Menu item — overrides equals()/hashCode() for cart grouping, toString()
Model/Order.java	Confirmed order entity — immutable fields, toString()
service/ICartOperations.java	Interface defining addToCart(), showCart(), placeOrder()
service/UserService.java	register() and login() — Regex validation, Base64 encoding
Service/ResturantService.java	getAllRestaurants(), getMenu(), getMenuItemById()
Service/OrderService.java	Cart lifecycle — addToCart(), getGroupedCart(), createFinalOrder(), getOrderHistory()
Exception/UserNotFoundException	Thrown by login() on invalid credentials
Exception/ItemNotAvailableException	Thrown by addToCart() on unavailable/missing item

6. DESIGN DIAGRAMS

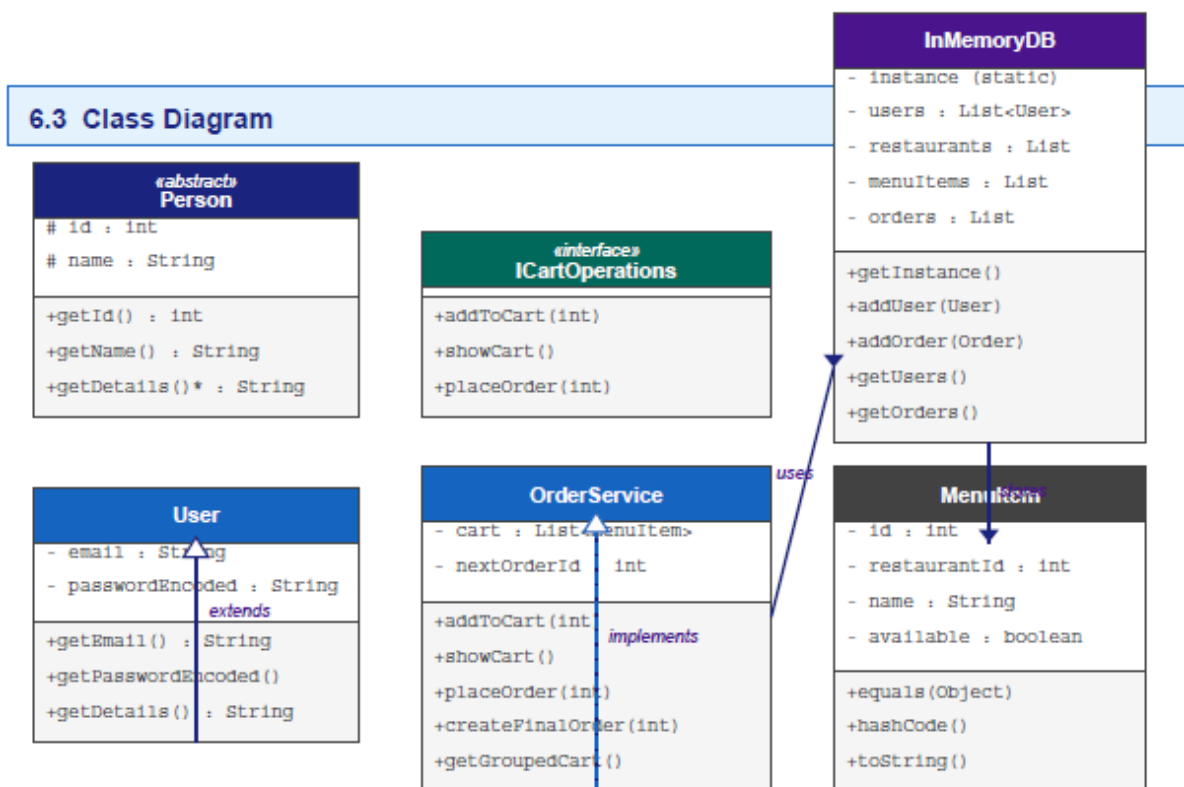
6.1 Use Case Diagram



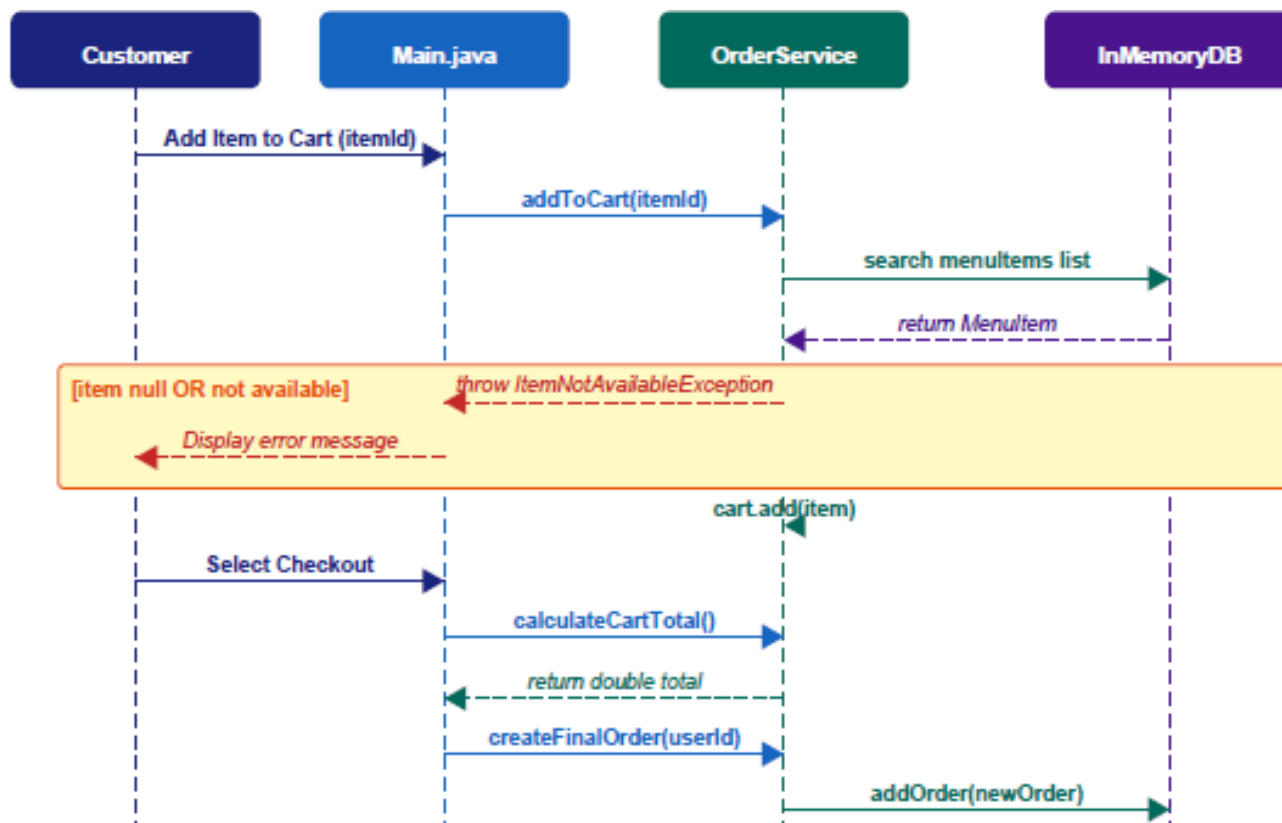
6.2 Process Flow / Workflow Diagram



6.3 Class Diagram



6.4 Sequence Diagram – Add to Cart & Checkout



7. IMPLEMENTATION DETAILS

Language	Java (JDK 17+)
Total Files	13 Java files across 4 packages + Main.java
Input Validation	java.util.regex.Pattern — email: <code>^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[a-z]{2,}\$</code>
Name Validation	java.util.regex.Pattern — <code>^[A-Za-z\s]{3,}\$</code>
Password Storage	java.util.Base64 encoding (simulation of production hashing)
Date/Time	java.time.LocalDateTime + DateTimeFormatter("dd MMM yyyy, hh:mm a")
Cart Grouping	Collectors.groupingBy(Function.identity(), Collectors.counting())
Total Calculation	Java Streams — <code>cart.stream().mapToDouble(MenuItem::getPrice).sum()</code>
Access Modifiers	All fields private/protected final — no public setters anywhere
DB Write Control	Only addUser() and addOrder() modify InMemoryDB; all getters are unmodifiable
Seed Data	3 restaurants, 7 menu items (1 intentionally unavailable), 1 pre-registered admin

8. Design Decisions & Rationale

- **8.1 The Singleton Pattern for Centralized Data Management** One of the most critical architectural decisions was the implementation of the Singleton design pattern for the `InMemoryDB` class. In a typical web application, a database acts as the single source of truth. Without a relational database, the system required a centralized data store that persists throughout the application's runtime. If multiple instances of the database class were instantiated by different services (e.g., `UserService` and `OrderService` each creating their own new `InMemoryDB()`), the data would immediately become desynchronized. To resolve this, the `InMemoryDB` constructor was made private, and a public static synchronized `getInstance()` method was created. The `synchronized` keyword ensures thread safety, guaranteeing that only one shared instance of the data store ever exists across the entire JVM lifecycle, preventing state inconsistencies.
- **8.2 Interface-Driven Architecture for Decoupling** To ensure the system is scalable and maintainable, the project strictly adheres to interface-driven design. The `ICartOperations` interface was established to define the contract for all cart-related activities (`addToCart`, `showCart`, `placeOrder`). The `OrderService` class then implements this interface. The rationale behind this decision is the principle of Abstraction—separating *what* the system does from *how* it does it. By having the presentation layer (`Main.java`) interact with the cart through its defined operations rather than a rigid implementation, the underlying logic can be entirely swapped out in the future. For example, if the system is upgraded to use a database-backed cart instead of an `ArrayList`, the `Main.java` UI code will not need a single line of modification.

- **8.3 Overriding Object Contracts (equals and hashCode)** A significant design challenge was automatically grouping identical menu items in the shopping cart to calculate quantities (e.g., preventing "Garlic Naan" from appearing on three separate lines when ordered three times). By default, Java's equals() method compares the memory addresses of objects. Therefore, two identical MenuItem objects added to the cart sequentially would be treated as completely distinct entities. To circumvent this, the equals() and hashCode() methods were explicitly overridden in the MenuItem.java model. The override forces the JVM to compare the unique id field of the menu items. This crucial design decision enabled the use of modern Java Streams, specifically Collectors.groupingBy(), to elegantly group identical items and automatically tally their quantities for the digital invoice.
- **8.4 Simulation of Production-Grade Security** While true cryptographic hashing (such as BCrypt or PBKDF2) falls outside the scope of this core Java project, storing user passwords in plain text is a severe anti-pattern. To demonstrate an understanding of enterprise security protocols, the java.util.Base64 encoder was implemented within the UserService. When a user registers, their plain-text password is converted into a Base64 encoded string before being saved to the InMemoryDB. During the login phase, the input password is similarly encoded, and the two encoded strings are compared. This simulates the one-way transformation process used in real-world authentication flows.

9. SCREENSHOTS / RESULTS

<pre>--- GUEST MENU --- 1. Register Account 2. User Login 3. Exit System Select an option (1-3): 1 --- ACCOUNT REGISTRATION --- Enter full name: red Enter email address: student@com [Error] Invalid format! Please provide a valid email. Enter email address: red@gmail.com Enter secure password (min 3 chars): red123 [Success] Account created successfully!</pre>		Regex rejecting "student@com" (invalid), then accepting "student@vit.edu"
<pre>--- MAIN DASHBOARD --- Welcome back, red! 1. View All Restaurants 2. Browse Restaurant Menu 3. Add Item to Cart 4. View Current Cart 5. Checkout & Place Order 6. View Order History 7. Secure Logout Select an option (1-7): █</pre>		ANSI-coloured authenticated main dashboard with box-drawing characters
<pre>--- MENU ITEMS --- ID Item Name Price ----- ----- 6 Margherita Pizza Rs.299.00 7 Cheesy Garlic Bread Rs.129.00 ----- -----</pre>		Tabular restaurant menu (only available items shown, Garlic Bread excluded)
<pre>=== YOUR SHOPPING CART === Qty Item Name Unit Price Total ----- ----- 2 Cheesy Garlic Bread Rs.129.00 Rs.258.00 ----- ----- SUBTOTAL: Rs.258.00</pre>		Same item added twice — cart shows Quantity: 2 (proves equals() override)
<pre>===== DIGITAL INVOICE ===== Order ID : #ORD-10001 Date : 17 Sept 2026, 01:22 am ----- 2x Cheesy Garlic Bread Rs.258.00 ----- TOTAL AMOUNT PAID : Rs.258.00 Order Status : CONFIRMED =====</pre>		Final invoice with LocalDateTime timestamp, grouped items, and total

10. TESTING APPROACH

Positive Test Cases

Test Scenario	Input	Expected Result
Valid Registration	Pratham / pratham@vit.edu / pass123	Account stored in InMemoryDB.users
Login	pratham@vit.edu + correct password	Session activated, dashboard shown
Add Different Items	Item IDs 1 and 4	Two separate line items in cart
Add Same Item Twice	Item ID 2, then Item ID 2 again	Cart shows "2x Garlic Naan Rs.80"
Confirm Checkout	Y at confirmation prompt	Invoice generated, cart cleared, order archived
Order History	Select option 6 after placing order	Completed order shown with correct total

Negative / Edge-Case Test Cases

Test Scenario	Input	Expected Behaviour
Invalid Email	"student@com" or "user"	Regex rejects — re-prompted, no crash
Invalid Name	"Us3r!" or "AB"	Regex rejects — letters only, 3+ chars
Text at number prompt	Type "abc" at menu	Scanner buffer cleared, gracefully retried
Wrong login	Correct email, wrong pass	UserNotFoundException caught, message shown
Unavailable item	Add Item ID 3 (Tandoori Roti)	ItemNotAvailableException caught, message shown
Abort checkout	N at confirmation prompt	Cart preserved intact, no order placed
Empty cart checkout	Select Checkout with no items	"Cart is empty" notice, returns to dashboard

11. Challenges Faced

11.1 The Scanner Buffer Infinite Crash Loop The most persistent technical challenge occurred within the presentation layer when handling user input via the `java.util.Scanner` class. When the terminal prompted the user for an integer (such as selecting a menu option or entering a restaurant ID), entering an alphabetic character or symbol caused the `Scanner.nextInt()` method to throw an `InputMismatchException`. Compounding the issue, the invalid token remained stuck in the Scanner's buffer, causing the `while(true)` menu loop to read the same invalid token repeatedly, resulting in an infinite crash loop. This was overcome by engineering a custom `getValidIntegerInput()` utility method. This method uses `sc.hasNextInt()` to check the token type before consuming it. If the token is invalid, the buffer is explicitly cleared using `sc.next()`, and the user is gracefully re-prompted, making the entire CLI entirely crash-proof.

11.2 Maintaining Encapsulation vs. Data Accessibility During the initial phases of integrating the `InMemoryDB` with the service classes, there was a strong temptation to make the `ArrayList` data structures public for easy access and modification. However, this violated the core OOP pillar of Encapsulation, leaving the application vulnerable to external classes arbitrarily clearing or altering user and order data. The challenge was providing the necessary read-and-write access to the services without exposing the internal lists. This was solved by making all lists private and exposing them only through `Collections.unmodifiableList()` in the getter methods. This ensures that services can iterate through the data to find what they need, but any structural modifications (like adding a new user or order) must pass through tightly controlled mutator methods (`addUser()`, `addOrder()`).

11.3 Cart State Management During Aborted Checkouts Managing the lifecycle of the shopping cart presented a logical hurdle. Initially, the cart was being cleared as soon as the user pressed "Checkout." However, if the user decided to abort the transaction at the final Y/N confirmation prompt, their carefully curated cart was lost, resulting in poor user experience. The logic had to be heavily refactored so that the `cart.clear()` execution was moved exclusively into the `createFinalOrder()` method inside `OrderService`. This ensured that the cart was only emptied *after* the order was successfully archived to the ledger, completely preserving the user's session data during aborted checkouts.

12. LEARNINGS & KEY TAKEAWAYS

- **OOP in Practice:** Building a real system made the four pillars concrete and purposeful. The abstract Person class, ICartOperations interface, private final fields, and Singleton pattern are all real design decisions — not textbook exercises.
- **Design Patterns Solve Real Problems:** The Singleton pattern solved a genuine data consistency problem. Understanding why a pattern exists — not just how to write it — is the most important takeaway.
- **Exceptions Are Communication:** Custom exceptions (UserNotFoundException, ItemNotAvailableException) make error handling explicit and readable. They communicate intent far better than returning null or -1.
- **Encapsulation Is Not Optional:** Making InMemoryDB fields public seemed convenient but broke the entire system design. Proper encapsulation with private fields and controlled accessors is not a formality — it prevents real bugs.
- **The Java Collections Framework Replaces Databases:** ArrayList and Streams can replace a SQL database at the right scale. Choosing the right tool for the right problem is a core software engineering skill.
- **UX Matters Even in Terminals:** ANSI colours, table formatting, screen pauses, and clear error messages transform a functional program into a usable one. Good software considers the user at every layer

13. Future Enhancements

13.1 Persistent Data Storage via File I/O Serialization Currently, the InMemoryDB relies on the Java Collections Framework, meaning all registered users, carts, and order histories are volatile and completely erased when the JVM terminates. A primary future enhancement will be the integration of Java's File I/O and Serialization APIs. By implementing the Serializable interface on the core model classes, the application will be able to export its state to .dat or .csv files upon shutdown. Upon application startup, a deserialization routine will read these files and repopulate the InMemoryDB, ensuring data persistence across multiple terminal sessions.

13.2 Graphical User Interface (GUI) Integration While the ANSI-formatted terminal provides a robust and professional developer experience, a commercial food management system requires an intuitive interface for non-technical users. The presentation layer (Main.java) has been entirely decoupled from the business logic layer, making the system perfectly primed for a UI overhaul. A future iteration will replace the console interactions with a modern JavaFX or Swing graphical user interface, featuring interactive buttons, scrollable tables for restaurant menus, and visual pop-up dialogs for exception handling and invoice generation.

13.3 Relational Database Connectivity (JDBC) To transition this project from an academic simulation to a production-ready enterprise application, the InMemoryDB singleton will be deprecated in favor of a true relational database management system (RDBMS) such as MySQL or PostgreSQL. This enhancement will utilize the Java Database Connectivity (JDBC) API to establish connections, execute raw SQL queries, and map result sets directly to the Java POJOs (Plain Old Java Objects). This will allow the system to handle massive concurrent read/write operations and execute complex relational queries, such as aggregating the most popular menu items across all restaurants.

13.4 Multi-Threading for Order Processing Currently, placing an order is a synchronous, blocking operation. To simulate the real-world delay of a kitchen preparing food, future versions of the application could implement Java Multi-threading. By wrapping the order confirmation process in a Thread or implementing the Runnable interface, the system could simulate an asynchronous "Preparation" phase, allowing the user to continue browsing the application while their order status dynamically updates from "PREPARING" to "READY" in the background.

14. REFERENCES

1. **Course Syllabus: CSE2006 – Programming in Java, VIT Bhopal University (2021)**
2. **VITyarthi Project Guidelines: "Build Your Own Project" submission document**
3. **Herbert Schildt, Java: The Complete Reference, 11th Edition, Oracle Press, 2018**
4. **Oracle Java Documentation – Collections Framework:
docs.oracle.com/javase/8/docs/technotes/guides/collections/**
5. **Oracle Java Documentation – java.util.regex.Pattern:
docs.oracle.com/en/java/docs/api/java.base/java/util/regex/Pattern.html**
6. **Oracle Java Documentation – java.time.LocalDateTime:
docs.oracle.com/en/java/docs/api/java.base/java/time/LocalDateTime.html**
7. **Oracle Java Documentation – java.util.Base64:
docs.oracle.com/en/java/docs/api/java.base/java/util/Base64.html**